Multimodal Crime / Incident Report Analyzer

AI-Powered Unified Incident Analysis System

AI for Engineers — Spring 2026
University of Houston

Lokeshwar Reddy Peddarangareddy — Audio Analyst & Integration Lead
Neha Reddy Poreddy — Document Analyst
Nagraj V Vallakati — Image Analyst
Rahul Sarma Vogeti — Video Analyst
Swet Vimalkumar Patel — Text Analyst

GitHub: https://github.com/Lokeshloki1219/Multimodal-Incident-Analyzer.git

1. Introduction

Every day, emergency response departments across the country deal with hundreds of incident reports coming in from different sources. A single traffic accident might generate a 911 call, a police report, photographs from the scene, surveillance camera footage, and social media posts from witnesses. The problem is that all of this information arrives in different formats — audio, PDFs, images, video, and text — and someone has to manually piece it all together to understand what actually happened. This takes time, creates errors, and slows down emergency response.

Our team was tasked with building a prototype AI system that can automatically process all five of these data types and merge them into a single, structured incident report. Each team member took ownership of one data modality and built an AI pipeline from scratch. Then we combined all five outputs into a unified dataset with severity classification, LLM-generated summaries, and an interactive dashboard.

This report walks through what we built, how we built it, what challenges we ran into, and what the final results look like. We tried to keep things practical — the system actually works end-to-end, and we can demonstrate it processing real data from each source.

2. System Architecture

The system follows the five-stage pipeline laid out in the assignment: data ingestion, AI processing per modality, information extraction, structured dataset generation, and a dashboard/query interface. Each stage feeds into the next, and the whole thing can run from a single terminal with five pipeline commands followed by the integration script.

The architecture diagram below shows how raw data from each source flows through its dedicated pipeline, produces a structured CSV, and then all five CSVs converge into the integration layer which produces the final unified incident dataset.

Figure 1: Pipeline Architecture Diagram

We chose this modular approach on purpose. Each student could develop and test their pipeline independently without waiting on anyone else. As long as everyone agreed on the CSV column names upfront, the integration step at the end would just work. In practice, we did have to iterate on the column names a couple of times when we noticed the assignment had two different column specifications (task text vs. expected output table), but we settled on including columns from both to be safe.

3. Individual Modality Pipelines

3.1 Audio Pipeline (Lokeshwar)

Dataset: 911 Calls + Wav2Vec2 from Kaggle — real emergency audio recordings with transcription data.

Models: OpenAI Whisper (base) for speech-to-text, spaCy (en_core_web_sm) for named entity recognition, HuggingFace distilbert for sentiment analysis.

Output: 703 rows in audio_output.csv with columns: Call_ID, Transcript, Extracted_Event, Location, Sentiment, Urgency_Score.

The audio pipeline starts by loading emergency call recordings and running them through OpenAI's Whisper model to convert speech to text. Whisper handles background noise and overlapping voices surprisingly well for a base model — most transcripts are usable even when the caller is distressed. After transcription, spaCy's NER model pulls out location mentions, person names, and organization references. We then run a HuggingFace sentiment classifier to determine whether the caller sounds calm or distressed, and map that to an urgency score between 0 and 1. Keywords like "fire," "shooting," and "trapped" also boost the urgency score.

One challenge was that the Kaggle dataset includes both the actual audio and pre-transcribed text. We used the pre-transcribed text for the bulk processing since running Whisper on 700+ audio files would take hours on CPU, but we verified Whisper's accuracy on a sample of 50 recordings to confirm the pipeline works end-to-end.

3.2 PDF Pipeline (Neha)

Dataset: Arkansas Police Department 1033 Training Plan Proposals from MuckRock — a 75-page FOIA-released PDF.

Models: PyMuPDF (fitz) for text extraction, pytesseract for OCR on scanned pages, spaCy for NER.

Output: 45 rows in pdf_output.csv with columns: Report_ID, Department, Incident_Type, Doc_Type, Date, Location, Program, Officer, Summary, Key_Detail.

This was probably the trickiest pipeline because the PDF turned out to be mostly scanned images. Out of 75 pages, only 14 had extractable digital text — the other 61 were scanned letterhead documents from various Arkansas sheriff's offices and police departments. PyMuPDF and pdfplumber both returned empty strings for those pages.

The fix was adding an OCR fallback directly inside the section-splitting function. When PyMuPDF returns fewer than 20 characters for a page, the pipeline renders the page as an image using PyMuPDF's get_pixmap() method and runs pytesseract on it. This approach avoids the need for the poppler library that pdf2image requires, making installation simpler. After OCR, spaCy extracts department names, dates, officer names, and program references. The section splitter detects document boundaries by looking for letterhead patterns, "To Whom It May Concern" headers, and empty page gaps between departments.

The result is 45 distinct report sections covering departments like Benton County Sheriff's Office, Cabot Police Department, Craighead County Sheriff's Office, Crawford County, Hot Springs PD, Independence County, Jacksonville PD, Jefferson County, Little Rock PD, Lonoke County, Mississippi County, Pulaski County, Rogers PD, and Union County.

3.3 Image Pipeline (Nagraj)

Dataset: Roboflow Fire Detection dataset (D-Fire) — fire and smoke images in YOLOv8 format.

Models: YOLOv8n fine-tuned on fire/smoke data, YOLOv8n pre-trained on COCO (80 object classes), ViT (google/vit-base-patch16-224) for scene classification, pytesseract for OCR.

Output: 100 rows in image_output.csv with columns: Image_ID, Scene_Type, Objects_Detected, Bounding_Boxes, Text_Extracted, Confidence_Score.

The image pipeline uses a dual-model approach. The primary model is a YOLOv8n that was fine-tuned on the D-Fire dataset to detect fire and smoke — the assignment's recommended dataset from Roboflow. The secondary model is a standard YOLOv8n pre-trained on COCO, which detects 80 common objects including people, vehicles, and some weapons. Running both models on the same image gives richer detections: a fire scene might show "fire, smoke, 2 persons, 1 truck" instead of just "fire, smoke."

Scene classification is handled by Google's ViT model, which categorizes each image as Fire Scene, Vehicle Accident, Assault Scene, or General Scene based on the visual content. The pipeline also runs pytesseract OCR on every image to extract any visible text like street signs or license plates, though most fire scene photographs don't contain readable text (86 out of 100 images had no extracted text, which is expected).

The training curves and confusion matrices from the fire model training are included in the repository under images/fire_detector/, showing that the model achieved good precision on fire and smoke detection.

3.4 Video Pipeline (Rahul)

Dataset: CAVIAR CCTV Dataset — simulated indoor surveillance footage from the University of Edinburgh.

Models: OpenCV for frame extraction, YOLOv8n for object detection, CLIP ViT (openai/clip-vit-base-patch32) for zero-shot scene classification.

Output: 198 rows in video_output.csv with columns: Clip_ID, Timestamp, Frame_ID, Event_Detected, Persons_Count, Objects, Confidence.

The video pipeline extracts frames from CCTV clips at regular intervals using OpenCV, then runs YOLOv8 on each extracted frame to detect people and objects. The interesting part is the event classification — instead of training a custom anomaly detection model, we used OpenAI's CLIP ViT model for zero-shot scene classification. CLIP can classify an image into categories it was never explicitly trained on by matching visual features to text descriptions. We feed it candidate labels like "person walking normally," "people fighting," "person collapsing," and "loitering" and it picks the most likely match.

The CAVIAR dataset has scenarios including people walking, meeting, fighting, and collapsing, which map well to the kind of events a crime/incident analyzer would need to detect. We processed multiple video clips and extracted 198 key frame analyses with timestamps, person counts, and event classifications.

3.5 Text Pipeline (Swet)

Dataset: CrimeReport from Kaggle — real crime text reports with crime type, location, and details.

Models: NLTK for text preprocessing, spaCy (en_core_web_sm) for NER, HuggingFace zero-shot classifier (facebook/bart-large-mnli) for topic classification, distilbert for sentiment analysis.

Output: 113 rows in text_output.csv with columns: Text_ID, Source, Raw_Text, Crime_Type, Location_Entity, Sentiment, Entities, Topic, Severity_Label.

The text pipeline processes crime news articles and social media posts. Raw text is first cleaned using NLTK — tokenization, stopword removal, and normalization. Then spaCy's NER model extracts named entities like people, locations, and organizations. The HuggingFace zero-shot classifier categorizes each text into crime types (Shooting, Robbery, Assault, Drug Crime, Law Enforcement, etc.) without needing any training data for those specific categories — it uses BART's natural language understanding to match text to labels.

Sentiment analysis tags each report as Positive, Negative, or Neutral, and a severity label (High, Medium, Low) is assigned based on the combination of crime type and sentiment. Shooting and assault reports with negative sentiment get High severity, while general law enforcement reports get Medium or Low.

4. Data Integration

The integration step is where the five separate pipelines come together. Since the five datasets come from completely unrelated sources (911 calls from across the US, Arkansas police PDFs, Roboflow fire images, Edinburgh CCTV footage, and Kaggle crime news), there's no natural way to match them by incident. The professor's own example in the assignment shows unrelated data under the same Incident_ID — "Building fire / trapped" from audio paired with "Robbery / Theft" from text under INC_001 — confirming that sequential alignment is the expected approach.

We assign Incident_IDs sequentially: row 1 from each CSV becomes INC_001, row 2 becomes INC_002, and so on. The five CSVs have different row counts (Audio: 703, PDF: 45, Image: 100, Video: 198, Text: 113), which naturally creates incidents with varying levels of data completeness:

Range	Modalities Available	Count
INC_001 – INC_045	Audio + PDF + Image + Video + Text	45
INC_046 – INC_100	Audio + Image + Video + Text	55
INC_101 – INC_113	Audio + Video + Text	13

INC_114 – INC_198	Audio + Video	85

Rows beyond INC_198 are dropped because single-modality rows don't demonstrate multimodal integration. The cutoff at 198 (matching the video row count) ensures every incident has at least two data sources.

4.1 Handling Missing Values

Missing modality data is filled with context-aware descriptions rather than generic NaN or "No data" placeholders. When an incident has no police report, the PDF_Doc_Type column reads "No police report filed" instead of being blank. Similarly, missing image data shows "No scene photograph available," missing video shows "No CCTV footage available," and so on. This makes the dataset human-readable and clearly communicates why data is absent rather than leaving the reader guessing.

Two metadata columns track data completeness: Sources_Available lists which modalities contributed (e.g., "Audio + Image + Video + Text") and Modality_Count gives the numeric count (e.g., 4 out of 5). These columns enable filtering in the dashboard.

4.2 Severity Classification

Severity is calculated adaptively based on whichever signals are available for each incident. Three modalities contribute severity signals: Audio provides an Urgency_Score (0 to 1), Image provides a Confidence_Score (0 to 1), and Text provides a Severity_Label mapped to numeric values (High = 1.0, Medium = 0.5, Low = 0.2). For complete incidents, all three signals are averaged. For partial incidents, only available signals are used — so an incident with just audio and video calculates severity from the audio urgency score alone, since video doesn't produce a severity signal. The averaged score maps to High (above 0.7), Medium (above 0.4), or Low (0.4 and below).

5. Bonus Features

5.1 Streamlit Dashboard

The interactive dashboard is built with Streamlit and loads the final_incidents.csv directly. It provides sidebar filters for severity level, modality count, and incident ID range. The main view shows key metrics (total incidents, high/medium/low counts, average modality coverage), severity distribution and modality coverage bar charts, top audio events and text crime types breakdowns, a color-coded incident table, and a per-incident detail view showing all modality data for a selected incident. Tabs at the bottom let users explore the raw data from each individual pipeline.

5.2 LLM-Based Summarization

The summarizer uses HuggingFace's google/flan-t5-base model, which runs entirely locally with no API key required. For each row in the final dataset, it constructs a prompt containing all

available modality fields and asks the model to generate a 2–3 sentence human-readable incident summary. The model downloads automatically on first run (about 990MB) and processes roughly 1–2 rows per second on CPU. The generated summaries are stored in an AI_Summary column appended to the CSV and displayed in the dashboard's detail view.

5.3 Real-Time Processing

The real-time monitor uses Python's watchdog library to watch a designated /new_data folder. When a new file is dropped in, the system detects its type from the file extension (.wav routes to audio, .pdf to PDF, .jpg to image, .mp4 to video, .csv to text), runs the appropriate pipeline, appends the result to the modality CSV, re-runs the integration merge, and generates an AI summary for just the new row. The Streamlit dashboard picks up changes on its next refresh cycle. This can be demonstrated live by dragging a file into the folder while the dashboard is running.

6. Results and Discussion

The final integrated dataset contains 198 incidents spanning all five modalities. Of these, 45 incidents have complete data from all five sources, 55 have four modalities, 13 have three, and 85 have two. The severity distribution across the full dataset shows a mix of High, Medium, and Low classifications, with the distribution weighted toward Medium due to the averaging approach.

The system successfully demonstrates the end-to-end pipeline from raw unstructured data to structured incident reports. Each modality contributes meaningful information: audio provides the event description and urgency level, PDFs provide document classification and department information, images provide object detection results with confidence scores, video provides timestamped event detection, and text provides crime type classification with sentiment analysis.

6.1 Challenges Faced

The biggest technical challenge was the PDF pipeline. The assignment's tip said "this is a text-based PDF so pdfplumber will extract it cleanly," but the actual LESO2.pdf turned out to be 81% scanned images. Our initial pipeline only extracted 6 rows from the 14 digital-text pages. Adding OCR inside the section splitter brought the count up to 45 sections covering all departments in the document.

Another challenge was the column specification conflict in the assignment. Each student's section has both a "task text" specification and an "expected output table" with different column names. For example, the PDF task says "Report_ID, Incident_Type, Date, Location, Officer, Summary" but the expected table shows "Report_ID, Department, Doc_Type, Date, Program, Key_Detail." We resolved this by including columns from both specifications, putting the expected table columns first since the Final Integration Table references them (PDF_Doc_Type and Text_Crime_Type both come from the expected tables).

The image pipeline required a dual-model approach because the assignment tasks ask for detecting “vehicles, fire, people, weapons, damage” but the recommended dataset only contains fire and smoke images. We addressed this by running both a fine-tuned fire detector and a COCO pre-trained model on each image.

6.2 AI Models Summary

Model	Purpose	Pipeline
OpenAI Whisper (base)	Speech-to-text transcription	Audio
spaCy en_core_web_sm	Named Entity Recognition	Audio, PDF, Text
YOLOv8n (COCO)	Object detection (80 classes)	Image, Video
YOLOv8n (fine-tuned)	Fire/smoke detection	Image
ViT (google)	Scene classification	Image
CLIP ViT (openai)	Zero-shot scene classification	Video
BART (facebook)	Zero-shot topic classification	Text
Flan-T5-base (google)	Incident summarization	Integration
pytesseract	OCR text extraction	PDF, Image
HuggingFace distilbert	Sentiment analysis	Audio, Text

7. Conclusion

We built a working multimodal incident analysis system that processes five types of unstructured data and produces a unified, structured incident report. The system uses 10 different AI models across the five pipelines, handles missing data gracefully, computes adaptive severity scores, generates natural language summaries, supports real-time file ingestion, and provides an interactive dashboard for exploring the results.

The project taught us practical lessons about dealing with messy real-world data (scanned PDFs, noisy audio, ambiguous text), managing column specifications when requirements are unclear, and building modular pipelines that can be developed independently and integrated at the end. The final dataset of 198 incidents with varying modality completeness demonstrates that the system works across a range of data availability scenarios — from fully complete five-source incidents to partial two-source incidents.

If we had more time, we would improve the PDF OCR accuracy, add more diverse image datasets beyond fire scenes, train a custom anomaly detection model for the video pipeline instead of relying on zero-shot classification, and deploy the dashboard to a cloud platform for remote access. But as a two-week prototype, the system demonstrates the core concept effectively: AI can automatically extract structured information from unstructured multimodal data and make it queryable for emergency response teams.